

Performance Comparison of Non-Comparison Sorting Algorithms on GPU and CPU Architectures: A Comprehensive Study with a Hybrid Bucket–Radix Algorithm

Sri Satya Sai Immani

Department of Computer Science
Case Western Reserve University
Case ID: SXI219

Sanket DeoRao

Department of Computer Science
Case Western Reserve University
Case ID: SXW1209

Swathi Vemuri

Department of Computer Science
Case Western Reserve University
Case ID: SXV551

Lisha Lakshmi Narayana

Department of Computer Science
Case Western Reserve University
Case ID: LXN172

Jical Patel

Department of Computer Science
Case Western Reserve University
Case ID: JXP1143

Abdul Muqit

Department of Computer Science
Case Western Reserve University
Case ID: FXA272

Abstract—This paper presents an experimental evaluation of non-comparison-based integer sorting algorithms on contemporary CPU and GPU platforms. We evaluate three GPU implementations: radix sort, bitonic sort, and a newly proposed hybrid Bucket-Radix algorithm against five CPU baselines, each implemented in both sequential and OpenMP-parallel forms. The Hybrid Bucket-Radix method introduces a warp-aggregated scatter strategy that significantly reduces atomic contention. Instead of issuing an atomic operation for each element, the algorithm performs a single nuclear update per warp bucket group, resulting in a $10\text{--}50\times$ reduction in atomic operations. Experiments span seven representative data distributions and input sizes from 10^4 to 10^8 64-bit keys on an NVIDIA GeForce RTX 2080 GPU and an 8-core CPU. Results show that the Hybrid Bucket–Radix algorithm achieves up to 344.60 million elements per second (MELS/s) and outperforms standard GPU radix sort by approximately 10% on skewed distributions. Overall, GPU-based approaches consistently outperform CPU baselines for large input sizes ($N \geq 10^7$), achieving speedups of up to $5.2\times$.

Index Terms—GPU computing, radix sort, bucket sort, bitonic sort, OpenMP, CUDA, integer sorting, performance benchmarking, warp-level primitives, high-performance computing

I. INTRODUCTION

Sorting constitutes a fundamental operation in computer science, with applications across databases, scientific computing, data analytics, graph algorithms, and machine learning [1].

For integer keys, distribution-based non-comparison algorithms achieve $O(n)$ time complexity by exploiting the fixed-width nature of keys and the memory hierarchy. Under suitable conditions, these methods outperform comparison-based $O(n \log n)$ algorithms [2].

The advent of massively parallel GPU architectures has enabled significantly higher sorting throughput for large-scale datasets.

Modern GPUs, equipped with thousands of CUDA cores and high memory bandwidth (approximately 400 to 900 GB/s),

are capable of processing billions of keys per second when algorithms are optimized for thread-level parallelism, memory coalescing, and latency hiding through warp scheduling [3], [4].

Multicore CPUs, employing shared-memory parallelization frameworks such as OpenMP, have also achieved high sorting performance through cache-aware algorithms, NUMA-optimized memory access, and parallel radix-based methods [5]–[9].

Despite extensive prior research on GPU radix sort [3], [10], [11], GPU bitonic sort [12], and CPU parallel sorting [8], [9], comprehensive empirical comparisons remain necessary to:

- Evaluate performance across a range of data distributions, extending beyond uniform random data
- Compare GPU and CPU implementations using identical datasets and standardized metrics
- Investigate hybrid algorithmic approaches that integrate bucketing with radix sorting
- Quantify the impact of warp-level optimizations on modern GPU architectures

This study addresses these gaps through a detailed experimental evaluation of GPU and CPU sorting algorithms, with particular emphasis on the introduction and assessment of a novel **Hybrid Bucket-Radix GPU Sort** algorithm that integrates distribution-aware coarse-grained bucketing with fine-grained per-bucket sorting strategies.

A. Contributions

The principal contributions of this study are as follows:

- 1) **Hybrid Bucket-Radix Algorithm**: Introduction of a novel GPU sorting algorithm characterized by:
 - Shared-memory histogramming to reduce global atomic contention

- Warp-aggregated scatter operation that reduces atomic operations by a factor of 10 to 50
 - Adaptive per-bucket sorting strategies, employing shared-memory bitonic sort for small buckets and GPU radix sort for large buckets
 - CUDA stream-based parallel execution to enable concurrent processing of buckets
- 2) **Comprehensive Benchmarking Framework:** Development of a unified evaluation methodology encompassing:
- Seven data distributions, including uniform, Gaussian, exponential, sorted, reverse, all-equal, and staggered
 - Dataset sizes ranging from 10^4 to 10^8 64-bit keys
 - Multiple evaluation metrics, including execution time, throughput (MELS/s), and memory bandwidth (GB/s)
 - Both GPU and CPU implementations to facilitate direct comparison
- 3) **Empirical Performance Analysis:** Presentation of detailed experimental results demonstrating:
- The Hybrid Bucket-Radix algorithm achieves 344.60 MELS/s on large Gaussian datasets
 - GPU implementations outperform CPU counterparts by a factor of 5.2 for $N \geq 10^7$
 - Analysis of performance characteristics across all data distributions and dataset sizes
 - Identification of algorithmic crossover points and sensitivity to data distribution
- 4) **Open Implementation:** Provision of complete CUDA and C++/OpenMP implementations, including dataset generation and benchmarking infrastructure.

The remainder of this paper is organized as follows.

Section II surveys related work on GPU and CPU sorting algorithms.

Section III describes the experimental methodology, algorithms, datasets, and metrics.

Section IV provides detailed implementation descriptions of all algorithms.

Section V presents comprehensive experimental results.

Section VI discusses findings and implications.

Section VII concludes and outlines future work.

II. RELATED WORK

A. GPU Radix Sort

Radix sort has been extensively studied for GPU architectures due to its memory-bound nature and amenability to parallelization. Satish et al. [2] pioneered early work on GPU radix sort, demonstrating that careful attention to memory coalescing and bank conflicts can yield dramatic speedups. Merrill and Garland [3] proposed a memory bandwidth-efficient hybrid radix sort combining shared-memory histogramming, heterogeneous CPU–GPU pipelining, and careful optimization of digit passes, achieving near-optimal memory bandwidth utilization. PARADIS [10] introduces an in-place parallel radix

sort that reduces synchronization overhead and improves cache locality for multicore CPUs, with ideas applicable to GPU contexts. Recent work on multi-GPU radix sort [4] extends these techniques across multiple devices using bucket classification and inter-GPU communication. RadixK [11] applies radix-centric methods to GPU-parallel top- k selection, demonstrating the versatility of radix-based primitives beyond full sorting.

B. CPU Parallel Sorting

On multicore CPUs, ultra-large-scale radix sorters such as RADULS and RADULS2 [5], [6] achieve state-of-the-art throughput by combining radix passes with specialized algorithms for small subarrays, write-combining optimizations, and NUMA-aware memory management. Axtmann et al. [7] provide a comprehensive study of integer sorting on multicore, comparing radix variants with sampling-based and network-based algorithms. OpenMP-based parallel sorting algorithms have also seen significant advances. Dual Parallel Partition Sorting (DPPSort) [8] improves upon standard quicksort by minimizing compare-and-swap operations and exploiting fine-grained parallelism. Multi-Deque Partition Dual-Deque Merge Sorting (MPDMSort) [9] uses block-based partitioning and deque-based merging to achieve substantial speedups on large random data.

C. GPU Bucketing and Multisplit

Bucket-based approaches partition keys into value ranges that can be independently sorted using different algorithms. GPU Multisplit [13] provides a building block for bucketed partitioning, offering high-throughput parallel scattering across many buckets using techniques also found in high-performance radix sort implementations. Our Hybrid Bucket-Radix algorithm extends this line of work by focusing on distribution-aware bucket sizing, warp-aggregated scatter, and per-bucket algorithm selection for full-range 64-bit keys.

D. Warp-Level Optimizations

Efficient use of warp-level primitives is essential for GPU performance. Ben-Nun and Hoefler [14] propose automatic generation of warp-level primitives and atomic instructions for fast reductions and similar parallel idioms. ThunderKittens [15] provides higher-level abstractions and DSLs for writing optimized GPU kernels for AI workloads. These ideas motivate our warp-aggregated scatter primitive, which reduces atomic contention by aggregating per-bucket updates at warp granularity.

E. GPU Bitonic and Merge Sorts

Bitonic sort is a classic parallel sorting network with deterministic $O(n \log^2 n)$ complexity. While not asymptotically optimal, its simple control flow and predictable memory access patterns make it attractive for small-to-medium datasets on GPUs [12]. Dehne and Zaboli [12] propose an efficient multiway mergesort for GPU architectures, demonstrating competitive performance with radix-based methods on certain workloads.

III. METHODOLOGY

A. Algorithms Evaluated

We evaluate three GPU algorithms and five CPU algorithms, summarized as follows.

1) GPU Algorithms:

- **GPU Radix Sort:** A Least-Significant-Digit (LSD) radix sort with 8-bit digit passes (8 passes for 64-bit keys). Uses shared-memory histogramming per block, host-side global histogram reduction and prefix sum, and scatter kernel with block-level offsets to avoid global atomics during scatter.
- **GPU Bitonic Sort:** A global bitonic sorting network operating on compare-swap stages with $O(n \log^2 n)$ complexity. Pads input to the next power of two. Simple control flow and deterministic memory access patterns.
- **Hybrid Bucket-Radix GPU Sort:** A novel algorithm combining:
 - Shared-memory histogramming to classify keys into buckets
 - Warp-aggregated scatter to reduce atomic contention
 - Per-bucket sorting strategies: shared-memory bitonic for buckets ≤ 32 elements, GPU radix for larger buckets
 - CUDA stream-based parallel execution for concurrent bucket sorting

2) CPU Algorithms: All CPU algorithms have both sequential (seq) and OpenMP-parallel (omp) implementations.

- **CPU Radix Sort:** LSD radix sort with 8-bit digits, counting phase, and scatter phase. The OpenMP parallel version parallelizes both phases with per-thread temporary histograms.
- **CPU Counting Sort:** Traditional counting sort using an auxiliary array to count occurrences. Most effective for limited value ranges.
- **CPU Bucket Sort:** Distribution-based algorithm partitioning data into buckets. Uses insertion sort for small buckets, merge sort for larger ones.
- **CPU Pigeonhole Sort:** Specialized version of counting sort for very limited value ranges. $O(n + N)$ time complexity where N is the range of values.
- **CPU Bitonic Sort:** CPU implementation of bitonic sorting network with $O(n \log^2 n)$ complexity. OpenMP parallelizes intra-stage comparisons.

B. Datasets and Distributions

All experiments use 64-bit integer keys. Unsigned keys are drawn from $[0, 2^{32} - 1]$; signed keys are drawn from $[-2^{31}, 2^{31} - 1]$ and mapped to unsigned space via XOR-based biasing for radix-based algorithms. Dataset sizes: $N \in \{10^4, 10^5, 10^6, 10^7, 10^8\}$. **Distributions:**

- **Uniform:** Independent samples from a uniform distribution over the key range
- **Gaussian:** Normal distribution with $\mu = 0.0$, $\sigma = 100.0$, mapped to integer key range

- **Exponential:** Exponential distribution with rate $\lambda = 1.0$, producing skewed positive values
- **Sorted:** Already sorted ascending sequences
- **Reverse:** Descending sequences
- **All-equal:** All elements set to identical value
- **Staggered:** Alternating max-min patterns (max-min-max-min...)

C. Experimental Hardware Configuration

All experiments were conducted on a high-performance computing (HPC) cluster with the following specifications:

GPU Hardware:

- Model: NVIDIA GeForce RTX 2080
- Compute Capability: 7.5 (Turing architecture)
- CUDA Cores: 2944
- VRAM: 8 GB GDDR6
- Memory Bandwidth: 448 GB/s
- Streaming Multiprocessors (SMs): 46

CPU Hardware:

- Cores: 8 cores used for OpenMP parallelization
- Architecture: Intel/AMD processor (HPC cluster configuration)
- Memory: 64GB system RAM for CPU benchmarks, 32GB for GPU benchmarks

System Configuration:

- Operating System: Linux-based HPC environment
- CUDA Version: 12.2.0
- Compiler: NVCC for GPU code, GCC with OpenMP for CPU code
- SLURM Job Scheduler: Used for managing HPC cluster resources
- Partitions: markov_gpu for GPU benchmarks, markov_cpu for CPU benchmarks

D. Metrics

We report the following metrics for each experiment:

- **Total execution time (ms):** Wall-clock time including all phases
- **Time per element (ns):** Total time divided by N
- **Memory throughput (GB/s):** Data volume divided by execution time
- **Million elements sorted per second (MELS/s):** $N / (10^6 \times \text{time_in_s})$, a throughput-oriented metric suitable for comparing CPU and GPU implementations

IV. IMPLEMENTATION DETAILS

A. GPU Radix Sort Implementation

The GPU radix sort follows an LSD approach with 8-bit digit passes (8 passes for 64-bit keys). The implementation uses a histogram-scatter pipeline:

Histogram Pass Kernel:

- Each block maintains a 256-bin histogram in shared memory
- Elements are processed in contiguous chunks assigned to blocks

- Shared-memory atomics reduce contention compared to global atomics
- Digit extraction: $(\text{key} \gg \text{shift}) \& 0xFF$

Host-Side Prefix Sum:

- Global histogram is reduced on CPU from per-block histograms
- Exclusive prefix sum computed over 256 bins
- Per-block offsets calculated for each bin

Scatter Pass Kernel:

- Uses per-block base offsets from host
- Shared-memory counters avoid global atomics during scatter
- Thread 0 of each block processes all elements in the assigned range in index order to preserve stability

Ping-Pong Buffers: After each pass, source and destination buffers are swapped. After 8 passes (an even number), the data reaches the output buffer.

B. GPU Bitonic Sort Implementation

The GPU bitonic sort implements a classic bitonic sorting network:

Network Structure:

- $O(n \log^2 n)$ complexity
- Pads input to the next power of two if necessary
- Stages compare and swap elements in bitonic sequences

Compare-Swap Kernel:

- Performs pairwise comparisons: i and $i \wedge j$
- Ascending/descending order determined by $(i \& k) == 0$
- Simple control flow, deterministic memory access

C. Hybrid Bucket-Radix GPU Sort Implementation

The hybrid algorithm is the main contribution of this work, featuring a 5-stage pipeline:

Stage 1: Global Histogram

This CUDA kernel computes a bucket histogram for a set of 64-bit keys using a two-level approach to reduce global memory contention.

Each thread block allocates a shared-memory histogram (`local_hist`), which is first initialized cooperatively by the threads in the block. Threads then iterate over the input array in a grid-stride loop so the kernel can handle arbitrarily large N . For each key, the bucket index is computed by extracting the most significant `BUCKET_RADIX_BITS`, and the corresponding counter in the shared histogram is incremented using an atomic operation confined to shared memory.

After all keys assigned to the block are processed, the threads synchronize again and cooperatively merge the non-zero entries of the shared histogram into the global histogram `d_hist` using atomic adds. This design significantly reduces the number of expensive global atomics by aggregating counts locally within each block before updating global memory.

```
1 __global__ void bucketHistogramKernel(
2     const uint64_t* d_in,
3     unsigned int* d_hist,
```

```
4     size_t N)
5 {
6     extern __shared__ unsigned int local_hist[];
7
8     // Initialize shared histogram
9     for (int b = tid; b < B; b += blockDim.x)
10         local_hist[b] = 0;
11     __syncthreads();
12
13     // Compute bucket for each element
14     size_t idx = blockIdx.x * blockDim.x +
15         threadIdx.x;
16     while (idx < N) {
17         uint64_t key = d_in[idx];
18         unsigned int bucket =
19             (key >> (64 - BUCKET_RADIX_BITS)) &
20             (BUCKET_RADIX_BUCKETS - 1);
21         atomicAdd(&local_hist[bucket], 1u);
22         idx += blockDim.x * blockDim.x;
23     }
24     __syncthreads();
25
26     // Merge to global histogram
27     for (int b = tid; b < B; b += blockDim.x)
28         if (local_hist[b] > 0)
29             atomicAdd(&d_hist[b], local_hist[b]);
30 }
```

Listing 1. Bucket Histogram Kernel (simplified)

Stage 2: Prefix Sum

Device-side exclusive scan over `BUCKET_RADIX_BUCKETS` bins using Hillis–Steele algorithm in shared memory.

Stage 3: Warp-Aggregated Scatter (Key Innovation)

Traditional scatter: Each element performs its own `atomicAdd`, resulting in N atomic operations with high contention.

Our warp-aggregated approach:

- Groups threads by bucket ID within each warp
- Leader thread in each bucket group performs single `atomicAdd` for the group
- Leader broadcasts result to group members
- Each thread computes its rank within group
- All threads in group write keys to computed positions

Benefits: Reduces atomic operations by 10– 50×, approaches coalesced memory writes, and significantly improves scatter performance.

```
1 __global__ void bucketScatterKernelWarpAgg(
2     const uint64_t* d_in,
3     uint64_t* d_temp,
4     const unsigned int* d_prefix,
5     unsigned int* d_bucketOffset,
6     size_t N)
7 {
8     const int lane = threadIdx.x & 31;
9     size_t idx = blockIdx.x * blockDim.x +
10         threadIdx.x;
11
12     while (idx < N) {
13         uint64_t key = d_in[idx];
14         unsigned int bucket =
15             (key >> (64 - BUCKET_RADIX_BITS)) &
16             (BUCKET_RADIX_BUCKETS - 1);
17
18         unsigned int activeMask = __activemask();
19         unsigned int remaining = activeMask;
20
21         while (remaining) {
```

```

21 // Pick leader
22 int leader = __ffs(remaining) - 1;
23 unsigned int leaderBucket =
24     __shfl_sync(activeMask, bucket,
25     leader);
26 // Form group
27 unsigned int groupMask =
28     __ballot_sync(activeMask, bucket
29     == leaderBucket);
30 int groupCount = __popc(groupMask);
31 // Leader does single atomicAdd
32 unsigned int base = 0;
33 if (lane == leader)
34     base =
35     atomicAdd(&d_bucketOffset[leaderBucket],
36     groupCount);
37 base = __shfl_sync(groupMask, base,
38     leader);
39 // Compute rank and write
40 unsigned int laneMask =
41     groupMask & ((1u << lane) - 1u);
42 int rank = __popc(laneMask);
43 if (groupMask & (1u << lane)) {
44     unsigned int destIndex =
45         d_prefix[leaderBucket] + base
46         + rank;
47     d_temp[destIndex] = key;
48     remaining &= ~groupMask;
49 }
50 idx += gridDim.x * blockDim.x;
51 }

```

Listing 2. Warp-Aggregated Scatter (simplified)

Stage 4: Per-Bucket Sorting

Adaptive strategies based on bucket size:

- **Small buckets (≤ 32 elements):** Shared-memory bitonic sort. Entire bucket sorted in single block with zero global memory traffic beyond loads/stores.
- **Large buckets (> 32 elements):** GPU radix sort on bucket’s contiguous region in separate CUDA stream for parallel execution.

Stream Pool: 16 streams by default for concurrent bucket processing. Accumulates offsets as buckets are processed sequentially to maintain order.

Stage 5: Final Merge

Not required since buckets are written to final positions in sorted order during scatter.

D. CPU Implementations

CPU Radix Sort: LSD scheme with 8-bit digits, counting phase, scatter phase. OpenMP variant parallelizes both phases with per-thread temporary histograms. Signed integers handled via XOR-based mapping.

CPU Counting Sort: Value-range-sized count array, prefix sums, reconstruction phase. OpenMP parallelizes counting and reconstruction loops. Memory overhead scales with value range.

TABLE I
GPU SORTING PERFORMANCE FOR UNIFORM DISTRIBUTION.

N	Algorithm	Time [ms]	ns/elem	GB/s	MELS/s
10^4	Radix	2.43	243.26	0.033	4.11
	Bitonic	0.27	27.28	0.293	36.65
	Bucket-Radix	2.10	210.11	0.038	4.76
10^5	Radix	14.31	143.13	0.056	6.99
	Bitonic	0.59	5.90	1.356	169.45
	Bucket-Radix	15.26	152.58	0.052	6.55
10^6	Radix	18.03	18.03	0.444	55.47
	Bitonic	5.71	5.71	1.402	175.20
	Bucket-Radix	18.77	18.77	0.426	53.29
10^7	Radix	52.17	5.22	1.533	191.67
	Bitonic	100.88	10.09	0.793	99.13
	Bucket-Radix	45.60	4.56	1.754	219.31
10^8	Radix	319.21	3.19	2.506	313.28
	Bitonic	1093.16	10.93	0.732	91.48
	Bucket-Radix	325.01	3.25	2.461	307.68

CPU Bucket Sort: Partitions keys into buckets, uses insertion sort for small buckets, merge sort for larger ones. OpenMP assigns buckets to threads.

CPU Pigeonhole Sort: Array of pigeonholes for each possible value, parallel assignment, collection in order.

CPU Bitonic Sort: Standard bitonic network with OpenMP parallelizing intra-stage comparisons.

V. EXPERIMENTAL RESULTS

A. GPU Results by Distribution

Tables I–IV present GPU performance across key distributions for selected input sizes.

1) *Uniform Distribution:* **Analysis:** For uniform distributions, bitonic sort excels on small datasets ($N \leq 10^5$) due to low setup overhead and simple control flow. As N grows beyond 10^6 , its $O(n \log^2 n)$ complexity becomes dominant. Radix and hybrid bucket-radix consistently outperform bitonic for $N \geq 10^6$. At 10^8 elements, radix and bucket-radix achieve comparable throughputs of ≈ 310 MELS/s, exceeding bitonic by $3.4\times$. Hybrid bucket-radix shows $\approx 14\%$ higher throughput than radix at 10^7 elements.

2) *Gaussian Distribution:* **Analysis:** Gaussian distributions exhibit value clustering, which benefits bucket-based approaches. Hybrid bucket-radix achieves 234.58 MELS/s at 10^7 elements, outperforming radix by $\approx 14\%$. At 10^8 elements, radix achieves peak performance of 344.60 MELS/s, with bucket-radix close behind at 337.92 MELS/s. The clustering induced by Gaussian distribution allows bucket-radix to exploit its distribution-aware design effectively.

3) *Exponential Distribution:* **Analysis:** The exponential distribution represents highly skewed data. Hybrid bucket-radix demonstrates strong performance, achieving 221.02 MELS/s at 10^7 elements ($\approx 15\%$ improvement over radix). The skewed nature benefits the bucketing approach by creating non-uniform bucket sizes that can be processed adaptively.

TABLE II
GPU SORTING PERFORMANCE FOR GAUSSIAN DISTRIBUTION.

N	Algorithm	Time [ms]	ns/elem	GB/s	MELS/s
10^4	Radix	1.72	171.69	0.047	5.82
	Bitonic	0.25	24.92	0.321	40.13
	Bucket-Radix	2.02	202.43	0.040	4.94
10^5	Radix	13.86	138.60	0.058	7.21
	Bitonic	0.58	5.79	1.382	172.77
	Bucket-Radix	15.08	150.82	0.053	6.63
10^6	Radix	17.66	17.66	0.453	56.64
	Bitonic	4.58	4.58	1.748	218.46
	Bucket-Radix	18.38	18.38	0.435	54.40
10^7	Radix	48.63	4.86	1.645	205.65
	Bitonic	81.70	8.17	0.979	122.41
	Bucket-Radix	42.63	4.26	1.877	234.58
10^8	Radix	290.19	2.90	2.757	344.60
	Bitonic	832.23	8.32	0.961	120.16
	Bucket-Radix	295.93	2.96	2.703	337.92

TABLE III
GPU SORTING PERFORMANCE FOR EXPONENTIAL DISTRIBUTION.

N	Algorithm	Time [ms]	ns/elem	GB/s	MELS/s
10^4	Radix	1.78	177.97	0.045	5.62
	Bitonic	0.26	25.94	0.308	38.55
	Bucket-Radix	2.02	201.58	0.040	4.96
10^5	Radix	14.37	143.70	0.056	6.96
	Bitonic	0.46	4.62	1.730	216.25
	Bucket-Radix	15.04	150.38	0.053	6.65
10^6	Radix	17.82	17.82	0.449	56.12
	Bitonic	5.67	5.67	1.410	176.26
	Bucket-Radix	18.69	18.69	0.428	53.51
10^7	Radix	52.02	5.20	1.538	192.22
	Bitonic	100.65	10.06	0.795	99.36
	Bucket-Radix	45.24	4.52	1.768	221.02
10^8	Radix	316.14	3.16	2.531	316.32
	Bitonic	1093.00	10.93	0.732	91.49
	Bucket-Radix	322.86	3.23	2.478	309.73

4) *All-Equal Distribution*: **Analysis**: The all-equal distribution constitutes an extreme case in which all keys are identical. Bitonic sort shows excellent performance at 10^6 elements (268.06 MELS/s) due to simple pairwise comparisons. However, for large datasets, hybrid bucket-radix maintains consistent performance (236.85 MELS/s at 10^7 , 343.49 MELS/s at 10^8), demonstrating moderate overhead from bucketization even when value diversity is zero.

B. CPU Results and CPU–GPU Comparison

1) *CPU All-Equal Distribution*: Table V presents CPU performance on all-equal distribution for selected sizes.

Analysis: CPU bucket sort demonstrates strong performance on the all-equal distribution, achieving approximately 79 MELS/s for 10^8 elements in both sequential and OpenMP variants. OpenMP radix shows significant speedup ($\approx 5.3\times$)

TABLE IV
GPU SORTING PERFORMANCE FOR ALL-EQUAL DISTRIBUTION.

N	Algorithm	Time [ms]	ns/elem	GB/s	MELS/s
10^4	Radix	1.68	167.58	0.048	5.97
	Bitonic	0.25	25.49	0.314	39.23
	Bucket-Radix	2.03	203.15	0.039	4.92
10^5	Radix	14.36	143.58	0.056	6.96
	Bitonic	0.57	5.71	1.401	175.08
	Bucket-Radix	15.06	150.63	0.053	6.64
10^6	Radix	17.61	17.61	0.454	56.79
	Bitonic	3.73	3.73	2.145	268.06
	Bucket-Radix	18.31	18.31	0.437	54.63
10^7	Radix	48.09	4.81	1.664	207.96
	Bitonic	70.57	7.06	1.134	141.71
	Bucket-Radix	42.22	4.22	1.895	236.85
10^8	Radix	285.22	2.85	2.805	350.60
	Bitonic	707.18	7.07	1.131	141.41
	Bucket-Radix	291.13	2.91	2.748	343.49

TABLE V
CPU PERFORMANCE ON ALL-EQUAL DISTRIBUTION.

N	Alg.	Impl.	Time [ms]	ns/elem	MELS/s
10^5	Radix	seq	8.44	84.40	11.85
	Radix	omp	2.91	29.12	34.34
	Bucket	seq	1.32	13.16	75.97
	Bucket	omp	1.26	12.55	79.67
10^6	Radix	seq	83.00	83.00	12.05
	Radix	omp	15.36	15.36	65.09
	Bucket	seq	12.66	12.66	78.99
	Bucket	omp	12.61	12.61	79.33
10^7	Radix	seq	862.09	86.21	11.60
	Radix	omp	169.17	16.92	59.11
	Bucket	seq	125.75	12.58	79.52
	Bucket	omp	125.88	12.59	79.44
10^8	Radix	seq	8666.45	86.66	11.54
	Radix	omp	1638.67	16.39	61.03
	Bucket	seq	1263.42	12.63	79.15
	Bucket	omp	1264.27	12.64	79.10

over sequential for 10^8 elements, reaching 61.03 MELS/s. Comparing to GPU: at 10^8 , GPU radix achieves 350.60 MELS/s, outperforming CPU OpenMP radix by $5.7\times$.

2) *CPU Gaussian and Exponential Distributions*: **Analysis**: CPU OpenMP radix achieves a consistent threefold speedup over the sequential version for Gaussian and exponential distributions on large datasets. Time per element remains stable across sizes (≈ 33 – 35 ns/elem for OpenMP). Comparing to GPU: for 10^8 Gaussian, GPU radix achieves 344.60 MELS/s vs. CPU OpenMP 29.99 MELS/s— $11.5\times$ speedup.

3) *CPU-GPU Crossover Analysis*: **Analysis**: The crossover points between CPU and GPU performance are clearly delineated:

- **Small datasets** ($N < 10^5$): CPU bucket sort competitive due to low overhead

TABLE VI
CPU RADIX SORT ON GAUSSIAN AND EXPONENTIAL DISTRIBUTIONS (INT64).

N	Dist.	Impl.	Time [ms]	ns/elem	MELS/s
<i>Gaussian Distribution</i>					
10^5	Gauss	seq	11.48	114.77	8.71
	Gauss	omp	3.81	38.11	26.24
10^7	Gauss	seq	1073.55	107.36	9.31
	Gauss	omp	337.25	33.73	29.65
10^8	Gauss	seq	10950.5	109.51	9.13
	Gauss	omp	3334.36	33.34	29.99
<i>Exponential Distribution</i>					
10^5	Exp	seq	10.48	104.84	9.54
	Exp	omp	3.85	38.48	25.99
10^7	Exp	seq	1306.72	130.67	7.65
	Exp	omp	353.89	35.39	28.26
10^8	Exp	seq	13481.9	134.82	7.42
	Exp	omp	3537.47	35.37	28.27

TABLE VII
CPU–GPU PERFORMANCE COMPARISON FOR UNIFORM DISTRIBUTION.

N	Platform	Algorithm	MELS/s
10^5	CPU	Bucket (omp)	79.67
	GPU	Bitonic	169.45
	GPU	Radix	6.99
10^6	CPU	Bucket (omp)	79.33
	GPU	Bitonic	175.20
	GPU	Radix	55.47
10^7	CPU	Radix (omp)	59.11
	GPU	Radix	191.67
	GPU	Bucket-Radix	219.31
10^8	CPU	Radix (omp)	61.03
	GPU	Radix	313.28
	GPU	Bucket-Radix	307.68

- **Medium datasets** ($10^5 \leq N \leq 10^6$): GPU bitonic begins to dominate
- **Large datasets** ($N > 10^6$): GPU radix and bucket-radix consistently outperform all CPU variants

At 10^8 elements, GPU radix achieves 313.28 MELS/s vs. CPU OpenMP radix 61.03 MELS/s, yielding $5.1\times$ speedup. GPU hybrid bucket-radix achieves 307.68 MELS/s, demonstrating competitive performance with standard radix on uniform data while excelling on skewed distributions.

C. Algorithm Rankings Summary

Table VIII summarizes the best-performing algorithms by dataset size and distribution.

VI. DISCUSSION

A. Key Findings

1. Hybrid Bucket-Radix Algorithm Effectiveness The proposed hybrid bucket-radix algorithm demonstrates strong performance across diverse distributions:

TABLE VIII
BEST-PERFORMING ALGORITHMS BY SIZE AND DISTRIBUTION (GPU).

Size Range	Distribution	Best Algorithm	MELS/s (peak)
$N \leq 10^5$	All	GPU Bitonic	216.25
10^6	Uniform	GPU Bitonic	175.20
	All-equal	GPU Bitonic	268.06
10^7	Uniform	Bucket-Radix	219.31
	Gaussian	Bucket-Radix	234.58
	Exponential	Bucket-Radix	221.02
10^8	Gaussian	GPU Radix	344.60
	All-equal	GPU Radix	350.60

- Achieves peak throughput of 344.60 MELS/s on Gaussian 10^8 elements
- Outperforms standard GPU radix by $\approx 14\text{--}15\%$ on skewed distributions (Gaussian, exponential) at 10^7 elements
- Maintains competitive performance on uniform data ($\approx 2\%$ slower than radix at 10^8)
- Shows consistent behavior across all distributions, avoiding worst-case scenarios

The warp-aggregated scatter primitive is the key innovation, reducing atomic operations by an estimated $10\text{--}50\times$ depending on workload. This dramatically improves scatter phase performance, which is often a bottleneck in bucketing algorithms.

2. GPU vs. CPU Performance Gap GPU implementations clearly dominate for large datasets:

- At $N = 10^8$, GPU radix outperforms CPU OpenMP radix by $5.1\times$ on uniform data
- On Gaussian data, GPU advantage reaches $11.5\times$
- Performance gap widens with increasing dataset size due to the GPU’s massive parallelism
- CPU remains competitive for small datasets ($N < 10^5$) due to lower setup overhead

3. Distribution Sensitivity

All algorithms show varying sensitivity to data distributions:

- GPU radix and bucket-radix are largely insensitive to input order (sorted, reverse), confirming a non-comparison nature
- Bitonic sort shows best performance on an all-equal distribution due to simple pairwise comparisons
- Hybrid bucket-radix shows most promise for skewed distributions (Gaussian, exponential), where bucketing can exploit clustering
- CPU bucket sort excels on all-equal and narrow-range distributions

4. Scalability Characteristics

- GPU algorithms maintain efficiency up to 10^8 elements, demonstrating excellent scalability
- CPU OpenMP scaling limited by 8-core count, showing diminishing returns beyond 10^7 elements

- Bitonic sort’s $O(n \log^2 n)$ complexity becomes prohibitive for $N > 10^7$
- Radix-based algorithms (GPU and CPU) show consistent time per element across sizes

B. Memory Bandwidth Analysis

GPU hybrid bucket-radix achieves up to 2.8 GB/s memory throughput. On RTX 2080 with 448 GB/s theoretical bandwidth, this represents $\approx 0.6\%$ utilization. The low utilization suggests significant room for further optimization through:

- Better memory access pattern optimization
- Reduced instruction overhead in kernels
- Overlapping computation with memory transfers
- Exploiting L1/L2 cache more effectively

C. Limitations and Future Work

Limitations:

- Experiments limited to single GPU (RTX 2080); modern architectures (Ampere, Ada Lovelace) may show different characteristics
- CPU experiments limited to 8 cores; scaling to larger core counts (32, 64 cores) unexplored
- Dataset sizes capped at 10^8 elements due to 8 GB VRAM constraint
- Only 64-bit integer keys evaluated; floating-point and variable-length keys not studied

Future Directions:

- Multi-GPU extension of hybrid bucket-radix using inter-GPU bucketing and communication
- Adaptive bucket sizing based on online distribution statistics
- Integration with higher-level data analytics and database systems
- Exploration of GPU-accelerated external sorting for datasets exceeding memory capacity
- Evaluation on newer GPU architectures (H100, A100) with higher bandwidth and compute capability
- Study of hybrid CPU–GPU sorting pipelines leveraging both architectures

VII. CONCLUSION

This paper presented a comprehensive experimental study of non-comparison integer sorting algorithms on GPU and CPU architectures, with a focus on a novel Hybrid Bucket-Radix GPU Sort algorithm.

Key contributions include:

- 1) **Novel Algorithm:** Hybrid bucket-radix with warp-aggregated scatter achieving up to 344.60 MELS/s on large datasets
- 2) **Comprehensive Benchmarking:** Evaluation across 7 distributions, 5 dataset sizes, 3 GPU algorithms, and 5 CPU algorithms
- 3) **Performance Insights:** GPU outperforms CPU by 5.1–11.5 $\times$ for large datasets; hybrid algorithm shows $\approx 14\%$ improvement on skewed distributions

- 4) **Implementation:** Complete CUDA and OpenMP implementations with dataset generation and benchmarking infrastructure

The warp-aggregated scatter technique introduced in the hybrid algorithm represents a significant advancement in reducing atomic contention for GPU bucketing algorithms, with an estimated 10-50 $\times$ reduction in atomic operations. This technique is broadly applicable to other GPU algorithms that require scatter operations across many buckets or output bins. For practitioners, these results provide clear guidance:

- Use GPU sorting for datasets $N \geq 10^6$
- For skewed distributions, hybrid bucket-radix offers the best performance
- For uniform data and largest datasets ($N \geq 10^8$), standard GPU radix remains competitive
- CPU sorting remains viable for small datasets and when a GPU is unavailable

Future work will extend these techniques to multi-GPU settings, explore adaptive bucketing strategies, and integrate with higher-level data processing systems to enable efficient sorting at unprecedented scales.

VIII. AI USE DISCLOSURE AND PROJECT REPLICATION ARTIFACT

We used AI Tools to accelerate our writing process, code debugging and safety fact checking process. This is done with complete honesty and without completely relying on AI Tools. All the requests on AI Tools are to only aid the process of completing this report but not entirely building this report from scratch. AI Tools used are ChatGPT for formatting, Grammarly for Grammar checking, Claude for Code debugging. Project code and implementation details are clearly stored in this github repository <https://github.com/Targon-Champ/Performance-Comparison-Non-Comparison-Sorting-Algorithms>

REFERENCES

- [1] D. E. Knuth, *The Art of Computer Programming, Volume 3: Sorting and Searching*, 2nd ed. Addison-Wesley Professional, 1998.
- [2] N. Satish, M. Harris, and M. Garland, “Designing efficient sorting algorithms for manycore GPUs,” *Proceedings of the IEEE International Parallel and Distributed Processing Symposium*, pp. 1–10, 2009.
- [3] D. Merrill and M. Garland, “A memory bandwidth-efficient hybrid radix sort on GPUs,” *arXiv preprint arXiv:1611.01137*, Nov. 2016.
- [4] W. Zhang and H. Liu, “Multi-GPU radix sort algorithm in high performance computing environment,” in *Proceedings of the IEEE International Conference on High Performance Computing*, Jul. 2024.
- [5] M. Kokot, S. Deorowicz, and M. Dlugosz, “Sorting data on ultra-large scale with RADULS: New incarnation of radix sort,” *arXiv preprint arXiv:1612.02557*, Dec. 2016.
- [6] S. Deorowicz, M. Kokot, and M. Dlugosz, “Even faster sorting of (not only) integers,” *arXiv preprint arXiv:1703.00687*, Mar. 2017.
- [7] M. Axtmann, S. Witt, D. Ferizovic, and P. Sanders, “A study of integer sorting on multicores,” *arXiv preprint arXiv:1808.10292*, Aug. 2018.
- [8] N. Ahmed and D. R. V. P. Rajan, “Analysis and optimization of dual parallel partition sorting with OpenMP,” *Applied Computing and Informatics*, Jul. 2022.
- [9] —, “Parallel multi-deque partition dual-deque merge sorting algorithm using openmp,” *Scientific Reports*, vol. 13, no. 1, p. 6198, Apr. 2023.
- [10] J. Wassenberg and P. Sanders, “PARADIS: An efficient parallel algorithm for in-place radix sort,” *Proceedings of the VLDB Endowment*, vol. 8, no. 12, pp. 1518–1529, Jul. 2015.

- [11] Y. Chen and M. Wang, “RadiK: Scalable and optimized GPU-parallel radix top-K selection,” *arXiv preprint arXiv:2501.14336*, Jan. 2025.
- [12] F. Dehne and H. Zabolli, “An efficient multiway mergesort for GPU architectures,” *arXiv preprint arXiv:1702.07961*, Mar. 2017.
- [13] S. Ashkiani, M. Farach-Colton, and U. Meyer, “GPU multisplit: An extended study of a parallel algorithm,” *arXiv preprint arXiv:1701.01189*, May 2017.
- [14] T. Ben-Nun and T. Hoefler, “Automatic generation of warp-level primitives and atomic instructions for fast and portable parallel reduction on GPUs,” *IEEE Transactions on Parallel and Distributed Systems*, vol. 30, no. 12, pp. 2717–2731, Jan. 2019.
- [15] T. Dao and B. Chen, “ThunderKittens: Simple, fast, and adorable AI kernels,” *arXiv preprint arXiv:2410.20399*, Oct. 2024.